

Đề tài Mini-project

Môn học: Lập trình và ảo hóa cho IoT - IT6130

Hình thức: Làm theo nhóm 3 sinh viên.

Công cụ chính: Docker, Docker Compose, Mosquitto MQTT, Python, InfluxDB, Grafana, FastAPI hoặc Flask.

Khung yêu cầu chung cho các đề tài

Mỗi đề tài yêu cầu sinh viên xây dựng một hệ thống IoT hoàn chỉnh ở mức phần mềm. Các thiết bị vật lý như sensor, actuator và gateway đều được giả lập bằng chương trình Python chạy trong container. Toàn bộ hệ thống phải được triển khai bằng Docker Compose.

Các thành phần bắt buộc

Mỗi nhóm phải tự lập trình và container hóa ít nhất các thành phần sau:

1. Một hoặc nhiều virtual sensor sinh dữ liệu định kỳ.
2. Một hoặc nhiều virtual actuator nhận lệnh điều khiển qua MQTT.
3. Một virtual IoT gateway hoặc edge controller xử lý dữ liệu, lưu trạng thái, phát hiện sự kiện và gửi lệnh điều khiển.
4. Một REST API để truy vấn trạng thái hệ thống và gửi lệnh thủ công.

Mỗi project phải sử dụng ít nhất các service nguồn mở sau:

- mosquitto: MQTT broker.
- influxdb: time-series database.
- grafana: dashboard monitoring.

Yêu cầu triển khai Docker

- Toàn bộ hệ thống phải chạy được bằng lệnh:

```
docker compose up -d --build
```

- Các service tự lập trình phải có Dockerfile riêng.
- Các thông số cấu hình phải đọc từ environment variables, không hard-code hoàn toàn trong source code.

- Các service trong container phải gọi nhau bằng service name, ví dụ mosquitto hoặc `http://influxdb:8086`, không dùng `localhost` để gọi service khác.
- Phải dùng Docker volume cho InfluxDB và Grafana.

Yêu cầu báo cáo và demo chung

Mỗi nhóm cần nộp source code, file `docker-compose.yml`, các Dockerfile, README, báo cáo PDF khoảng 15–20 trang, ảnh chụp màn hình. Báo cáo phải trình bày kiến trúc, topic MQTT, message format, logic xử lý, cách triển khai Docker Compose, dashboard và phân công công việc.

Đề tài 1. Virtual Smart Building Gateway

1. Tên đề tài

Xây dựng hệ thống Virtual IoT Gateway cho Smart Building với xử lý dữ liệu, phát hiện bất thường và điều khiển thiết bị qua MQTT

2. Bối cảnh

Trong các hệ thống IoT thực tế, đặc biệt là smart building, gateway không chỉ chuyển tiếp dữ liệu từ cảm biến lên cloud. Gateway còn có thể thực hiện các chức năng quan trọng như chuẩn hóa dữ liệu, lọc dữ liệu lỗi, phát hiện bất thường, ra quyết định cục bộ và gửi lệnh điều khiển xuống actuator.

Ví dụ, trong một tòa nhà thông minh, nhiều phòng có thể có cảm biến nhiệt độ, độ ẩm, ánh sáng, mức CO₂, cảm biến chuyển động và các actuator như quạt, đèn, hoặc cảnh báo. Gateway cần thu thập dữ liệu từ nhiều phòng, phát hiện trạng thái bất thường như nhiệt độ quá cao hoặc CO₂ vượt ngưỡng, sau đó gửi lệnh điều khiển tự động.

Mini-project này yêu cầu sinh viên xây dựng một hệ thống IoT hoàn chỉnh ở mức phần mềm, không cần thiết bị phần cứng thật. Các cảm biến, actuator và gateway đều được giả lập bằng các chương trình Python chạy trong container. Toàn bộ hệ thống phải được triển khai bằng Docker Compose.

3. Mục tiêu của mini-project

Sau khi hoàn thành mini-project, nhóm sinh viên cần đạt được các mục tiêu sau:

1. Lập trình được nhiều virtual IoT devices mô phỏng các phòng khác nhau trong một smart building.
2. Xây dựng được một virtual IoT gateway có khả năng nhận dữ liệu MQTT từ nhiều thiết bị.
3. Thiết kế được message format và topic hierarchy hợp lý.
4. Chuẩn hóa dữ liệu thô từ các thiết bị thành định dạng dữ liệu thống nhất.
5. Phát hiện được một số bất thường đơn giản bằng rule engine.
6. Gửi lệnh điều khiển tự động đến actuator simulator qua MQTT.
7. Lưu dữ liệu thô, dữ liệu chuẩn hóa, event bất thường và trạng thái actuator vào InfluxDB.
8. Xây dựng dashboard Grafana để quan sát dữ liệu, cảnh báo và trạng thái hệ thống.
9. Cung cấp REST API đơn giản để truy vấn trạng thái mới nhất của các phòng và gửi lệnh điều khiển thủ công.
10. Container hóa toàn bộ hệ thống và triển khai bằng Docker Compose.

4. Mô tả bài toán

Nhóm sinh viên cần xây dựng một hệ thống smart building ảo gồm ít nhất 3 phòng:

- room-01
- room-02
- room-03

Mỗi phòng có một virtual sensor node gửi dữ liệu định kỳ qua MQTT. Dữ liệu tối thiểu gồm:

- nhiệt độ;
- độ ẩm;
- cường độ ánh sáng;
- mức CO₂;
- trạng thái có người hay không.

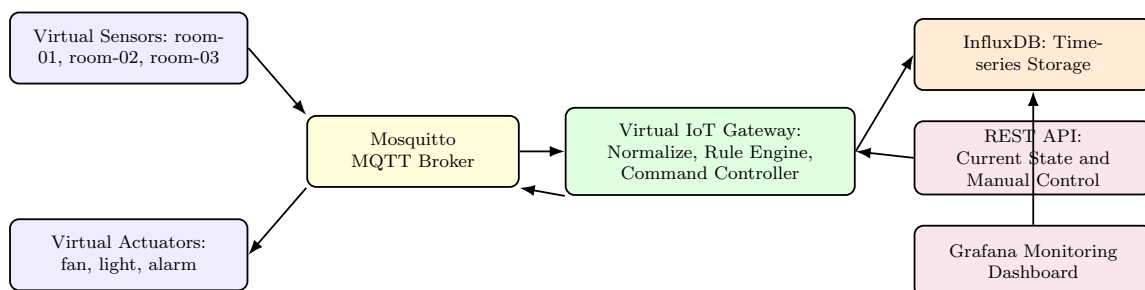
Mỗi phòng cũng có một virtual actuator node, ví dụ:

- quạt;
- đèn;
- cảnh báo.

Virtual IoT gateway phải nhận dữ liệu từ các sensor node, xử lý dữ liệu và gửi lệnh điều khiển đến actuator node khi phát hiện tình huống bất thường.

5. Kiến trúc hệ thống yêu cầu

Kiến trúc hệ thống tối thiểu như Hình 1.



Hình 1: Kiến trúc tổng quát của mini-project nâng cao

6. Các service bắt buộc

File `docker-compose.yml` phải triển khai tối thiểu các service sau:

- `mosquitto`: MQTT broker.
- `virtual-sensor-room-01`: sensor giả lập cho phòng 1.
- `virtual-sensor-room-02`: sensor giả lập cho phòng 2.
- `virtual-sensor-room-03`: sensor giả lập cho phòng 3.
- `virtual-actuator-room-01`: actuator giả lập cho phòng 1.
- `virtual-actuator-room-02`: actuator giả lập cho phòng 2.
- `virtual-actuator-room-03`: actuator giả lập cho phòng 3.
- `iot-gateway`: virtual IoT gateway.
- `influxdb`: time-series database.
- `grafana`: dashboard.
- `gateway-api`: REST API để xem trạng thái và điều khiển thủ công.

Nhóm có thể gộp hoặc tách service tùy thiết kế, nhưng phải giải thích rõ trong báo cáo. Tuy nhiên, bắt buộc phải có ít nhất các nhóm chức năng sau: sensor simulator, actuator simulator, MQTT broker, gateway, database, dashboard và API.

7. Thiết kế MQTT topic

Nhóm cần thiết kế topic hierarchy rõ ràng. Gợi ý:

7.1. Topic cho telemetry thô từ sensor

```
building/room-01/sensor/telemetry
building/room-02/sensor/telemetry
building/room-03/sensor/telemetry
```

7.2. Topic cho lệnh điều khiển actuator

```
building/room-01/actuator/command
building/room-02/actuator/command
building/room-03/actuator/command
```

7.3. Topic cho trạng thái actuator

```
building/room-01/actuator/status
building/room-02/actuator/status
building/room-03/actuator/status
```

7.4. Topic cho dữ liệu gateway đã chuẩn hóa

```
building/room-01/gateway/normalized  
building/room-02/gateway/normalized  
building/room-03/gateway/normalized
```

7.5. Topic cho event bất thường

```
building/room-01/gateway/event  
building/room-02/gateway/event  
building/room-03/gateway/event
```

8. Message format yêu cầu

8.1. Telemetry message từ sensor

Mỗi virtual sensor phải publish message JSON có dạng tối thiểu:

```
{  
  "device_id": "sensor-room-01",  
  "room_id": "room-01",  
  "temperature": 29.5,  
  "humidity": 73.2,  
  "light_lux": 380.0,  
  "co2_ppm": 950.0,  
  "occupancy": true,  
  "timestamp": "2026-06-10T10:00:00Z"  
}
```

8.2. Command message từ gateway đến actuator

Gateway gửi lệnh điều khiển xuống actuator bằng MQTT:

```
{  
  "room_id": "room-01",  
  "target": "fan",  
  "action": "on",  
  "reason": "temperature_high",  
  "timestamp": "2026-06-10T10:00:05Z"  
}
```

Các action tối thiểu:

- on
- off

Các target tối thiểu:

- fan
- light
- alarm

8.3. Status message từ actuator

Sau khi nhận lệnh, actuator phải publish lại trạng thái:

```
{
  "device_id": "actuator-room-01",
  "room_id": "room-01",
  "fan": "on",
  "light": "off",
  "alarm": "off",
  "last_command_reason": "temperature_high",
  "timestamp": "2026-06-10T10:00:06Z"
}
```

8.4. Event message từ gateway

Khi phát hiện bất thường, gateway phải sinh event:

```
{
  "room_id": "room-01",
  "event_type": "temperature_high",
  "severity": "warning",
  "value": 32.5,
  "threshold": 30.0,
  "action_taken": "fan_on",
  "timestamp": "2026-06-10T10:00:05Z"
}
```

9. Yêu cầu lập trình

9.1. Virtual Sensor

Nhóm cần lập trình virtual sensor bằng Python.

Yêu cầu:

1. Đọc cấu hình từ environment variables, ví dụ ROOM_ID, DEVICE_ID, MQTT_BROKER, PUBLISH_INTERVAL.
2. Sinh dữ liệu môi trường theo thời gian.
3. Dữ liệu phải có biến động hợp lý, không chỉ random hoàn toàn độc lập.
4. Có cơ chế sinh tình huống bất thường theo xác suất hoặc theo chu kỳ, ví dụ nhiệt độ tăng cao hoặc CO2 vượt ngưỡng.
5. Publish message JSON lên topic tương ứng của từng phòng.

Gợi ý: sinh viên có thể mô phỏng xu hướng bằng cách lưu giá trị trước đó và thay đổi nhẹ ở mỗi chu kỳ.

9.2. Virtual Actuator

Nhóm cần lập trình virtual actuator bằng Python.

Yêu cầu:

1. Subscribe topic command của phòng tương ứng.
2. Parse command JSON.
3. Cập nhật trạng thái nội bộ của fan, light, alarm.
4. Publish lại trạng thái lên topic status.
5. Ghi log khi nhận lệnh hợp lệ hoặc lệnh sai định dạng.

9.3. Virtual IoT Gateway

Đây là thành phần quan trọng nhất của mini-project.

Gateway phải thực hiện các chức năng sau:

1. Subscribe dữ liệu telemetry từ tất cả các phòng.
2. Validate message: kiểm tra room_id, device_id, timestamp và các field dữ liệu.
3. Chuẩn hóa message thành format thống nhất.
4. Lưu trạng thái mới nhất của từng phòng.
5. Ghi dữ liệu telemetry vào InfluxDB.
6. Phát hiện bất thường bằng rule engine.
7. Sinh event khi có bất thường.
8. Ghi event vào InfluxDB.
9. Publish event lên MQTT topic.
10. Gửi command đến actuator nếu cần.

9.4. Rule Engine

Gateway phải có ít nhất 4 luật xử lý:

1. Nếu `temperature > 30`, bật quạt.
2. Nếu `temperature < 27`, tắt quạt.
3. Nếu `co2_ppm > 1200`, bật alarm.
4. Nếu `occupancy = false` và `light_lux > 300`, tắt đèn.

Nhóm có thể bổ sung luật khác, ví dụ:

- Nếu độ ẩm quá cao, sinh cảnh báo.

- Nếu sensor không gửi dữ liệu trong một khoảng thời gian, sinh event `sensor_offline`.
- Nếu actuator không phản hồi sau khi nhận command, sinh event `actuator_no_response`.

9.5. REST API

Nhóm cần xây dựng REST API đơn giản, có thể dùng FastAPI.

API tối thiểu cần có các endpoint sau:

```
GET /health
GET /rooms
GET /rooms/{room_id}/state
GET /rooms/{room_id}/events
POST /rooms/{room_id}/command
```

Ý nghĩa:

- GET `/health`: kiểm tra API còn chạy hay không.
- GET `/rooms`: trả về danh sách phòng.
- GET `/rooms/{room_id}/state`: trả về trạng thái mới nhất của một phòng.
- GET `/rooms/{room_id}/events`: trả về các event gần nhất.
- POST `/rooms/{room_id}/command`: gửi lệnh điều khiển thủ công đến actuator.

Ví dụ body của API gửi lệnh thủ công:

```
{
  "target": "fan",
  "action": "on",
  "reason": "manual_control"
}
```

API có thể giao tiếp với gateway bằng một trong hai cách:

1. API publish command trực tiếp lên MQTT.
2. API đọc/ghi trạng thái thông qua một file, Redis, hoặc một cơ chế lưu trữ đơn giản do nhóm tự thiết kế.

Nhóm phải giải thích rõ lựa chọn thiết kế trong báo cáo.

10. Yêu cầu lưu trữ InfluxDB

Nhóm cần lưu ít nhất 3 loại dữ liệu vào InfluxDB:

1. **Telemetry data**: dữ liệu sensor sau khi chuẩn hóa.
2. **Event data**: các event bất thường do gateway sinh ra.
3. **Actuator status**: trạng thái fan, light, alarm của từng phòng.

Gợi ý measurement:

- room_telemetry
- gateway_events
- actuator_status

Các tag nên có:

- room_id
- device_id
- event_type
- severity

Các field nên có:

- temperature
- humidity
- light_lux
- co2_ppm
- occupancy
- fan
- light
- alarm

11. Yêu cầu Grafana Dashboard

Dashboard Grafana phải có ít nhất các panel sau:

1. Nhiệt độ theo thời gian cho từng phòng.
2. CO2 theo thời gian cho từng phòng.
3. Độ ẩm theo thời gian cho từng phòng.
4. Trạng thái fan/light/alarm theo từng phòng.
5. Số lượng event bất thường theo thời gian.
6. Bảng event gần nhất, gồm room_id, event_type, severity, action_taken.

Dashboard cần thể hiện được mối liên hệ giữa dữ liệu sensor, event gateway và trạng thái actuator.

Ví dụ: khi nhiệt độ phòng 1 vượt ngưỡng, dashboard cần cho thấy nhiệt độ tăng, event temperature_high xuất hiện và trạng thái fan chuyển sang on.

12. Yêu cầu Docker và ảo hóa

Toàn bộ hệ thống phải chạy bằng Docker Compose.

12.1. Dockerfile

Các service do nhóm tự lập trình phải có Dockerfile riêng:

- virtual_sensor/Dockerfile
- virtual_actuator/Dockerfile
- iot_gateway/Dockerfile
- gateway_api/Dockerfile

12.2. Environment variables

Không hard-code các thông số triển khai trong source code. Các service cần đọc cấu hình từ environment variables.

Ví dụ:

- MQTT_BROKER
- MQTT_PORT
- ROOM_ID
- DEVICE_ID
- PUBLISH_INTERVAL
- INFLUXDB_URL
- INFLUXDB_TOKEN
- INFLUXDB_ORG
- INFLUXDB_BUCKET

12.3. Network

Các service phải giao tiếp với nhau qua Docker Compose network.

Trong container, không dùng localhost để gọi service khác. Ví dụ:

```
MQTT_BROKER=mosquitto
INFLUXDB_URL=http://influxdb:8086
```

12.4. Volume

Nhóm phải dùng Docker volume cho:

- InfluxDB data.
- Grafana data.

Khuyến khích dùng volume hoặc bind mount cho Mosquitto config.

13. Cấu trúc thư mục gợi ý

Nhóm có thể tổ chức project như sau:

```
smart-building-iot-gateway/  
|-- docker-compose.yml  
|-- README.md  
|-- report.pdf  
|-- .env.example  
|-- mosquitto/  
|   '-- config/  
|       '-- mosquitto.conf  
|-- virtual_sensor/  
|   |-- sensor.py  
|   |-- requirements.txt  
|   '-- Dockerfile  
|-- virtual_actuator/  
|   |-- actuator.py  
|   |-- requirements.txt  
|   '-- Dockerfile  
|-- iot_gateway/  
|   |-- gateway.py  
|   |-- rule_engine.py  
|   |-- state_store.py  
|   |-- requirements.txt  
|   '-- Dockerfile  
|-- gateway_api/  
|   |-- api.py  
|   |-- requirements.txt  
|   '-- Dockerfile  
|-- docs/  
|   |-- architecture.png  
|   '-- topic-design.md  
'-- screenshots/  
    |-- docker-compose-ps.png  
    |-- mqtt-telemetry.png  
    |-- mqtt-command.png  
    |-- influxdb-data.png  
    '-- grafana-dashboard.png
```

14. Yêu cầu README

File README.md phải đủ rõ để người khác chạy lại project.

README cần có:

1. Mô tả ngắn gọn hệ thống.
2. Sơ đồ kiến trúc hoặc mô tả luồng dữ liệu.
3. Danh sách service.
4. Cách chạy hệ thống.
5. Cách kiểm tra log.
6. Cách truy cập Grafana, InfluxDB và REST API.

7. Cách gửi lệnh điều khiển thủ công.
8. Các lỗi thường gặp và cách khắc phục.

Các lệnh tối thiểu cần có trong README:

```
docker compose up -d --build
docker compose ps
docker compose logs -f iot-gateway
docker compose logs -f gateway-api
docker compose down
```

15. Yêu cầu demo

Khi demo, nhóm cần chứng minh các nội dung sau:

1. Chạy toàn bộ stack bằng Docker Compose.
2. Có ít nhất 3 virtual sensor đang publish dữ liệu.
3. Có ít nhất 3 virtual actuator đang nhận command.
4. Gateway nhận được telemetry từ nhiều phòng.
5. Gateway phát hiện ít nhất một event bất thường.
6. Gateway gửi được command tự động đến actuator.
7. Actuator publish lại status sau khi nhận command.
8. Dữ liệu được ghi vào InfluxDB.
9. Grafana dashboard hiển thị telemetry, event và actuator status.
10. REST API trả về trạng thái mới nhất của phòng.
11. REST API có thể gửi lệnh điều khiển thủ công đến actuator.

16. Câu hỏi bắt buộc trong báo cáo

Báo cáo cần trả lời các câu hỏi sau:

1. Vì sao hệ thống cần một IoT gateway thay vì để từng sensor gửi trực tiếp dữ liệu đến database?
2. Gateway trong project của nhóm thực hiện những chức năng nào?
3. Nhóm thiết kế MQTT topic hierarchy như thế nào? Vì sao?
4. Nhóm chuẩn hóa dữ liệu sensor như thế nào?
5. Rule engine của nhóm gồm những luật nào?
6. Khi gateway phát hiện bất thường, luồng xử lý từ telemetry đến command diễn ra như thế nào?

7. Docker Compose network giúp các service giao tiếp với nhau ra sao?
8. Vì sao trong container không nên dùng `localhost` để gọi service khác?
9. Nhóm sử dụng Docker volume cho những service nào? Vì sao?
10. Nếu muốn triển khai hệ thống này lên một IoT gateway thật, nhóm cần thay đổi những gì?

17. Phân công gợi ý cho nhóm 3 sinh viên

Nhóm có thể phân công như sau:

- **Thành viên 1:** lập trình virtual sensor, virtual actuator, thiết kế topic và message format.
- **Thành viên 2:** lập trình virtual IoT gateway, rule engine, ghi dữ liệu vào InfluxDB.
- **Thành viên 3:** lập trình REST API, cấu hình Docker Compose, Grafana dashboard, README và hỗ trợ tích hợp.

Lưu ý: cả nhóm phải cùng tham gia tích hợp và debug hệ thống. Báo cáo cần nêu rõ đóng góp của từng thành viên.

18. Tiêu chí chấm điểm

Nội dung	Điểm
Thiết kế kiến trúc Virtual IoT Gateway hợp lý	1.0
Thiết kế MQTT topic và message format rõ ràng, có tính mở rộng	1.0
Virtual sensor sinh dữ liệu hợp lý và có tình huống bất thường	1.0
Virtual actuator nhận command và publish status đúng	1.0
Gateway validate, normalize và lưu trạng thái phòng chính xác	1.2
Rule engine phát hiện bất thường và gửi command tự động	1.2
Ghi được telemetry, event và actuator status vào InfluxDB	1.0
REST API hoạt động đúng cho truy vấn trạng thái và điều khiển thủ công	0.8
Grafana dashboard thể hiện rõ telemetry, event và trạng thái actuator	0.8
Docker Compose, Dockerfile, network, volume và environment variables đúng	1.0
README, báo cáo, demo và giải thích hệ thống	1.0
Tổng	10.0

19. Yêu cầu nâng cao để cộng điểm khuyến khích

Nhóm có thể thực hiện thêm một số chức năng nâng cao:

1. Cấu hình username/password cho Mosquitto MQTT broker.
2. Dùng file `.env` để quản lý cấu hình.
3. Thêm health check cho các service trong Docker Compose.
4. Thêm cơ chế phát hiện sensor offline.
5. Thêm cơ chế actuator acknowledgement timeout.
6. Thêm API endpoint để thay đổi threshold của rule engine.
7. Thêm Grafana alert rule.
8. Thêm unit test cho rule engine.
9. Thêm script tự động publish dữ liệu bất thường để kiểm thử.
10. Mô phỏng mất kết nối MQTT và cơ chế reconnect.

20. Ràng buộc

- Không yêu cầu mua sensor, gateway hoặc actuator thật.
- Không sử dụng dịch vụ cloud trả phí.
- Không sử dụng phần mềm thương mại yêu cầu license trả phí.
- Hệ thống phải chạy được trên laptop cá nhân.
- Các service chính phải chạy bằng Docker Compose.
- Nhóm phải tự lập trình ít nhất bốn thành phần: virtual sensor, virtual actuator, virtual IoT gateway và REST API.
- Không được chỉ sửa lại trực tiếp bài labwork Buổi 2. Project phải có thêm logic gateway, rule engine, command/control và API.

21. Gợi ý kiểm tra nhanh trước khi nộp

Trước khi nộp bài, nhóm nên kiểm tra:

1. Chạy được toàn bộ hệ thống:

```
docker compose up -d --build
```

2. Tất cả container ở trạng thái running:

```
docker compose ps
```

3. Có telemetry từ ít nhất 3 phòng:

```
building/room-01/sensor/telemetry  
building/room-02/sensor/telemetry  
building/room-03/sensor/telemetry
```

4. Có command từ gateway đến actuator:

```
building/room-01/actuator/command
```

5. Có status từ actuator:

```
building/room-01/actuator/status
```

6. Có event bất thường:

```
building/room-01/gateway/event
```

7. InfluxDB có dữ liệu trong các measurement:

```
room_telemetry  
gateway_events  
actuator_status
```

8. Grafana dashboard hiển thị được dữ liệu mới theo thời gian.

9. REST API trả về trạng thái phòng.

10. REST API gửi được command thủ công.

22. Kết luận

Mini-project này yêu cầu sinh viên xây dựng một hệ thống IoT ảo hóa có tính thực tế cao hơn so với labwork cơ bản. Sinh viên không chỉ triển khai MQTT, InfluxDB và Grafana, mà còn phải lập trình một Virtual IoT Gateway có khả năng xử lý dữ liệu, phát hiện bất thường, điều khiển actuator và cung cấp REST API.

Thông qua đề tài này, sinh viên vận dụng đồng thời các kiến thức về lập trình IoT, giao thức MQTT, container hóa, Docker Compose, time-series database, dashboard monitoring và thiết kế kiến trúc phần mềm nhiều service. Đây là nền tảng quan trọng để tiếp tục học các nội dung nâng cao hơn ở Buổi 3 như vận hành, giám sát, bảo mật và mở rộng hệ thống IoT ảo hóa tại edge.

Đề tài 2. Virtual Smart Agriculture Gateway

1. Tên đề tài

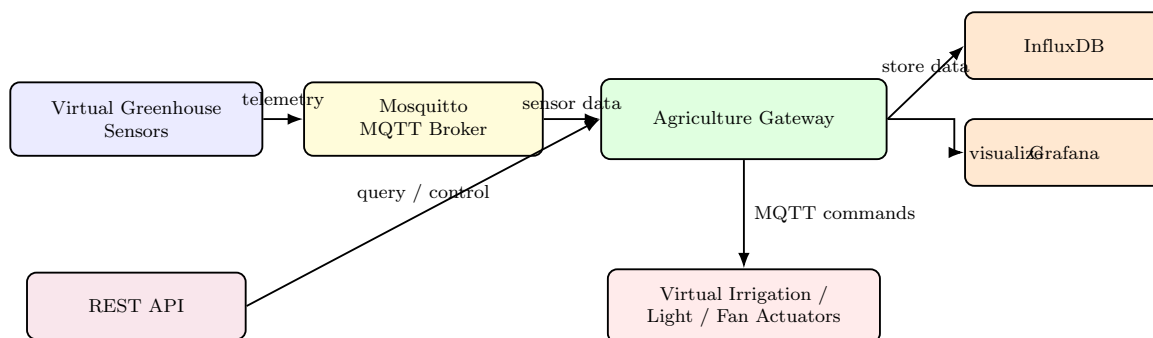
Xây dựng hệ thống Virtual Smart Agriculture Gateway cho giám sát nhà kính và điều khiển tưới tiêu tự động qua MQTT

2. Bối cảnh và mục tiêu

Trong nông nghiệp thông minh, hệ thống IoT thường thu thập dữ liệu từ nhiều cảm biến như nhiệt độ, độ ẩm không khí, độ ẩm đất, ánh sáng, pH đất và trạng thái bể nước. Gateway tại edge có nhiệm vụ xử lý dữ liệu cục bộ, phát hiện điều kiện bất thường và điều khiển actuator như bơm nước, quạt thông gió hoặc đèn chiếu sáng.

Đề tài này yêu cầu sinh viên xây dựng một hệ thống nhà kính ảo gồm nhiều khu vực trồng cây. Mỗi khu vực có sensor node và actuator node giả lập. Virtual agriculture gateway phải thu thập dữ liệu, chuẩn hóa dữ liệu, phát hiện bất thường, điều khiển tưới tiêu và lưu dữ liệu vào InfluxDB để hiển thị trên Grafana.

3. Kiến trúc hệ thống



Hình 2: Kiến trúc hệ thống của Đề tài 2

4. Các service bắt buộc

File `docker-compose.yml` cần có ít nhất các service sau:

- `mosquitto`: MQTT broker.
- `sensor-zone-01`, `sensor-zone-02`, `sensor-zone-03`: sensor giả lập cho 3 khu vực trồng cây.
- `actuator-zone-01`, `actuator-zone-02`, `actuator-zone-03`: actuator giả lập cho từng khu vực.
- `agri-gateway`: gateway xử lý dữ liệu và điều khiển tự động.
- `agri-api`: REST API để xem trạng thái và gửi lệnh thủ công.
- `influxdb`: lưu dữ liệu time-series.
- `grafana`: dashboard giám sát.

5. MQTT topic gợi ý

```
greenhouse/zone-01/sensor/telemetry
greenhouse/zone-02/sensor/telemetry
greenhouse/zone-03/sensor/telemetry

greenhouse/zone-01/actuator/command
greenhouse/zone-02/actuator/command
greenhouse/zone-03/actuator/command

greenhouse/zone-01/actuator/status
greenhouse/zone-02/actuator/status
greenhouse/zone-03/actuator/status

greenhouse/zone-01/gateway/event
greenhouse/zone-02/gateway/event
greenhouse/zone-03/gateway/event
```

6. Message format yêu cầu

Telemetry message từ sensor:

```
{
  "device_id": "sensor-zone-01",
  "zone_id": "zone-01",
  "air_temperature": 31.2,
  "air_humidity": 68.5,
  "soil_moisture": 24.1,
  "light_lux": 820.0,
  "soil_ph": 6.4,
  "water_tank_level": 42.0,
  "timestamp": "2026-06-12T10:00:00Z"
}
```

Command message từ gateway đến actuator:

```
{
  "zone_id": "zone-01",
  "target": "water_pump",
  "action": "on",
  "reason": "soil_moisture_low",
  "timestamp": "2026-06-12T10:00:05Z"
}
```

Status message từ actuator:

```
{
  "device_id": "actuator-zone-01",
  "zone_id": "zone-01",
  "water_pump": "on",
  "fan": "off",
  "grow_light": "off",
  "last_command_reason": "soil_moisture_low",
  "timestamp": "2026-06-12T10:00:06Z"
}
```

7. Logic gateway và rule engine

Gateway phải có ít nhất 5 luật xử lý:

1. Nếu `soil_moisture` nhỏ hơn 30, bật `water_pump`.
2. Nếu `soil_moisture` lớn hơn 45, tắt `water_pump`.
3. Nếu `air_temperature` lớn hơn 34, bật `fan`.
4. Nếu `light_lux` nhỏ hơn 300 trong trạng thái ban ngày giả lập, bật `grow_light`.
5. Nếu `water_tank_level` nhỏ hơn 15, sinh event `water_tank_low` và không bật bơm.

Gateway phải lưu trạng thái mới nhất của từng khu vực, ghi telemetry, event và actuator status vào InfluxDB, đồng thời publish event lên MQTT topic.

8. REST API tối thiểu

```
GET /health
GET /zones
GET /zones/zone-01/state
GET /zones/zone-01/events
POST /zones/zone-01/command
```

Endpoint POST command phải cho phép bật hoặc tắt thủ công `water_pump`, `fan` hoặc `grow_light`.

9. Dashboard Grafana

Dashboard phải có ít nhất các panel:

1. Nhiệt độ không khí theo từng zone.
2. Độ ẩm đất theo từng zone.
3. Mức nước bể chứa.
4. Trạng thái bơm, quạt và đèn.
5. Số lượng event bất thường theo thời gian.
6. Bảng event gần nhất.

10. Tiêu chí chấm điểm riêng

Nội dung	Điểm
Thiết kế kiến trúc smart agriculture gateway hợp lý	1.0
Sensor sinh dữ liệu có xu hướng và tình huống bất thường	1.0
Actuator nhận command và publish status đúng	1.0
Gateway validate, normalize, lưu state và phát hiện event đúng	1.5
Rule engine điều khiển tưới, quạt, đèn hợp lý	1.2
InfluxDB và Grafana thể hiện rõ telemetry, event, actuator status	1.2
REST API hoạt động đúng	0.8
Docker Compose, Dockerfile, network, volume, environment variables đúng	1.0
README, báo cáo, demo và giải thích tốt	1.3
Tổng	10.0

Đề tài 3. Virtual Cold-chain Monitoring Gateway

1. Tên đề tài

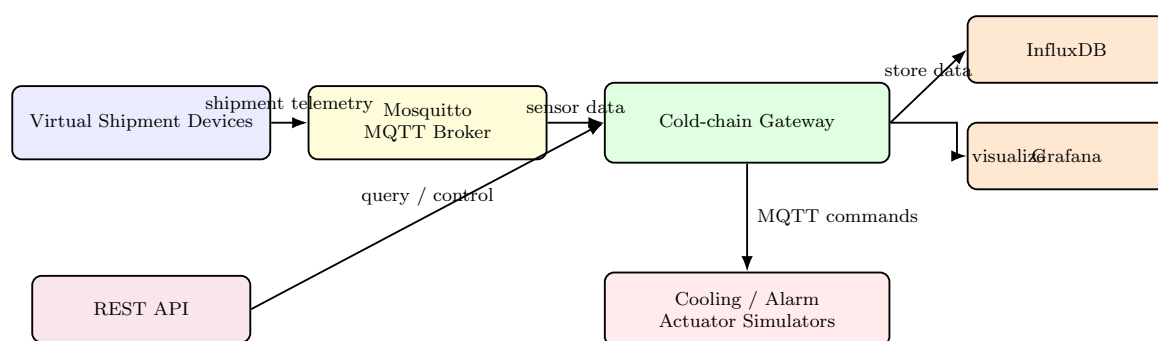
Xây dựng hệ thống Virtual Cold-chain Monitoring Gateway cho giám sát vận chuyển lạnh và cảnh báo vi phạm nhiệt độ qua MQTT

2. Bối cảnh và mục tiêu

Chuỗi cung ứng lạnh được sử dụng để vận chuyển vaccine, thực phẩm đông lạnh, mẫu y tế hoặc hàng hóa nhạy cảm với nhiệt độ. Trong quá trình vận chuyển, hệ thống IoT cần giám sát nhiệt độ, độ ẩm, trạng thái cửa khoang lạnh, pin thiết bị, vị trí giả lập và phát hiện các vi phạm như nhiệt độ vượt ngưỡng quá lâu hoặc cửa bị mở bất thường.

Đề tài này yêu cầu sinh viên xây dựng một gateway ảo cho cold-chain monitoring. Các xe lạnh hoặc thùng lạnh được mô phỏng bằng virtual device. Gateway phải theo dõi nhiều shipment, phát hiện vi phạm điều kiện bảo quản, sinh cảnh báo và gửi lệnh điều khiển đến actuator giả lập như cooling unit hoặc alarm.

3. Kiến trúc hệ thống



Hình 3: Kiến trúc hệ thống của Đề tài 3

4. Các service bắt buộc

- mosquitto: MQTT broker.
- shipment-device-01, shipment-device-02, shipment-device-03: thiết bị giám sát lô hàng giả lập.
- cooling-actuator-01, cooling-actuator-02, cooling-actuator-03: actuator giả lập.
- coldchain-gateway: gateway xử lý dữ liệu và phát hiện vi phạm.
- coldchain-api: REST API.
- influxdb: lưu dữ liệu time-series.
- grafana: dashboard giám sát.

5. MQTT topic gợi ý

```
coldchain/shipment-01/device/telemetry
coldchain/shipment-02/device/telemetry
coldchain/shipment-03/device/telemetry

coldchain/shipment-01/actuator/command
coldchain/shipment-02/actuator/command
coldchain/shipment-03/actuator/command

coldchain/shipment-01/actuator/status
coldchain/shipment-02/actuator/status
coldchain/shipment-03/actuator/status

coldchain/shipment-01/gateway/event
coldchain/shipment-02/gateway/event
coldchain/shipment-03/gateway/event
```

6. Message format yêu cầu

Telemetry message:

```
{
  "device_id": "device-shipment-01",
  "shipment_id": "shipment-01",
  "temperature": 7.8,
  "humidity": 63.0,
  "door_open": false,
  "battery_percent": 76.5,
  "latitude": 21.0278,
  "longitude": 105.8342,
  "timestamp": "2026-06-12T10:00:00Z"
}
```

Command message:

```
{
  "shipment_id": "shipment-01",
  "target": "cooling_unit",
  "action": "increase_power",
  "reason": "temperature_violation",
  "timestamp": "2026-06-12T10:00:05Z"
}
```

Status message:

```
{
  "device_id": "cooling-actuator-01",
  "shipment_id": "shipment-01",
  "cooling_unit": "high",
  "alarm": "on",
  "last_command_reason": "temperature_violation",
  "timestamp": "2026-06-12T10:00:06Z"
}
```

7. Logic gateway và rule engine

Gateway phải có ít nhất 5 luật xử lý:

1. Nếu temperature lớn hơn 8 trong 3 chu kỳ liên tiếp, sinh event temperature_violation.
2. Nếu temperature lớn hơn 8, gửi command increase_power đến cooling_unit.
3. Nếu door_open = true quá 2 chu kỳ liên tiếp, bật alarm.
4. Nếu battery_percent nhỏ hơn 20, sinh event battery_low.
5. Nếu không nhận telemetry từ một shipment trong một khoảng thời gian cấu hình, sinh event device_offline.

Gateway phải có cơ chế lưu trạng thái theo shipment để kiểm tra điều kiện kéo dài nhiều chu kỳ, không chỉ kiểm tra từng message độc lập.

8. REST API tối thiểu

```
GET /health
GET /shipments
GET /shipments/shipment-01/state
GET /shipments/shipment-01/events
POST /shipments/shipment-01/command
```

API phải hỗ trợ gửi lệnh thủ công như bật alarm, tắt alarm, tăng công suất cooling unit hoặc đưa cooling unit về chế độ normal.

9. Dashboard Grafana

Dashboard phải có ít nhất các panel:

1. Nhiệt độ theo từng shipment.
2. Độ ẩm theo từng shipment.
3. Trạng thái cửa khoang lạnh.
4. Battery level.
5. Số lần temperature violation.
6. Trạng thái cooling unit và alarm.
7. Bảng event gần nhất.

10. Tiêu chí chấm điểm riêng

Nội dung	Điểm
Kiến trúc cold-chain gateway hợp lý	1.0
Thiết kế telemetry, status và command message rõ ràng	1.0
Thiết bị shipment giả lập được tình huống vi phạm	1.0
Gateway theo dõi trạng thái qua nhiều chu kỳ	1.4
Rule engine phát hiện temperature violation, door open, battery low	1.4
Actuator nhận command và phản hồi status đúng	1.0
InfluxDB và Grafana thể hiện dữ liệu và event rõ ràng	1.2
REST API hoạt động đúng	0.8
Docker Compose và README tốt	1.2
Tổng	10.0

Đề tài 4. Virtual Industrial Machine Health Gateway

1. Tên đề tài

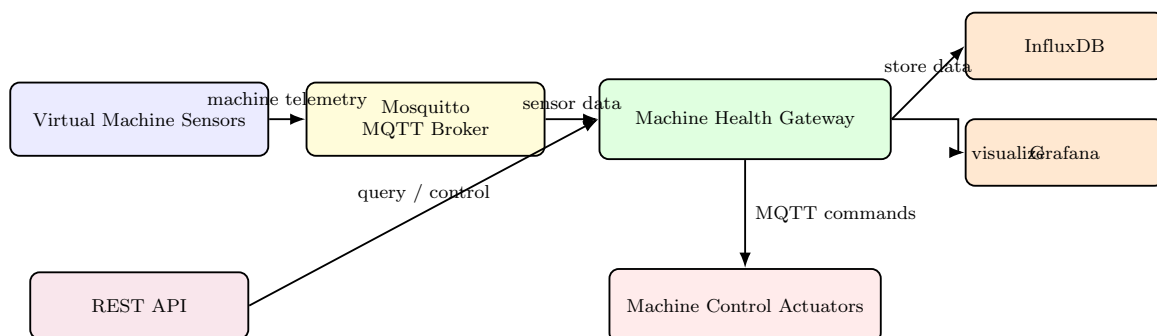
Xây dựng hệ thống Virtual Industrial Machine Health Gateway cho giám sát máy công nghiệp và phát hiện dấu hiệu hỏng hóc qua MQTT

2. Bối cảnh và mục tiêu

Trong môi trường sản xuất công nghiệp, các máy móc như motor, băng chuyền, máy nén hoặc robot cần được giám sát liên tục. Dữ liệu thường gồm rung động, nhiệt độ, tốc độ quay, dòng điện, trạng thái vận hành và mã lỗi. Edge gateway có thể xử lý dữ liệu gần nguồn, phát hiện bất thường và gửi lệnh dừng máy hoặc cảnh báo để tránh hỏng hóc nghiêm trọng.

Đề tài này yêu cầu sinh viên xây dựng một hệ thống giám sát sức khỏe máy công nghiệp ảo. Mỗi machine simulator phát dữ liệu telemetry qua MQTT. Gateway phải tính toán chỉ số sức khỏe đơn giản, phát hiện bất thường và gửi lệnh đến actuator simulator như machine_stop, cooling_on hoặc alarm_on.

3. Kiến trúc hệ thống



Hình 4: Kiến trúc hệ thống của Đề tài 4

4. Các service bắt buộc

- mosquitto: MQTT broker.
- machine-sensor-01, machine-sensor-02, machine-sensor-03: machine sensor giả lập.
- machine-actuator-01, machine-actuator-02, machine-actuator-03: actuator điều khiển máy giả lập.
- machine-gateway: gateway xử lý condition monitoring.
- machine-api: REST API.
- influxdb: lưu dữ liệu time-series.
- grafana: dashboard giám sát.

5. MQTT topic gợi ý

```
factory/machine-01/sensor/telemetry
factory/machine-02/sensor/telemetry
factory/machine-03/sensor/telemetry

factory/machine-01/actuator/command
factory/machine-02/actuator/command
factory/machine-03/actuator/command

factory/machine-01/actuator/status
factory/machine-02/actuator/status
factory/machine-03/actuator/status

factory/machine-01/gateway/event
factory/machine-02/gateway/event
factory/machine-03/gateway/event
```

6. Message format yêu cầu

Telemetry message:

```
{
  "device_id": "sensor-machine-01",
  "machine_id": "machine-01",
  "vibration_rms": 2.8,
  "temperature": 71.5,
  "rpm": 1450,
  "current_ampere": 8.2,
  "operation_mode": "running",
  "error_code": "none",
  "timestamp": "2026-06-12T10:00:00Z"
}
```

Gateway event:

```
{
  "machine_id": "machine-01",
  "event_type": "vibration_high",
  "severity": "critical",
  "health_score": 42.0,
  "action_taken": "alarm_on",
  "timestamp": "2026-06-12T10:00:05Z"
}
```

Command message:

```
{
  "machine_id": "machine-01",
  "target": "machine_controller",
  "action": "stop",
  "reason": "critical_health_score",
  "timestamp": "2026-06-12T10:00:06Z"
}
```

7. Logic gateway và rule engine

Gateway phải tính một `health_score` đơn giản trong khoảng từ 0 đến 100. Nhóm tự thiết kế công thức nhưng phải giải thích trong báo cáo. Ví dụ, health score giảm khi vibration, temperature hoặc current tăng vượt ngưỡng.

Gateway phải có ít nhất 5 luật xử lý:

1. Nếu `vibration_rms` lớn hơn 4.0, sinh event `vibration_high`.
2. Nếu `temperature` lớn hơn 80, bật `cooling`.
3. Nếu `current_ampere` lớn hơn 12, sinh event `over_current`.
4. Nếu `health_score` nhỏ hơn 50, bật `alarm`.
5. Nếu `health_score` nhỏ hơn 30, gửi command dừng máy.

8. REST API tối thiểu

```
GET /health
GET /machines
GET /machines/machine-01/state
GET /machines/machine-01/events
POST /machines/machine-01/command
```

API phải cho phép gửi lệnh thủ công như `start`, `stop`, `cooling_on`, `cooling_off`, `alarm_on`, `alarm_off`.

9. Dashboard Grafana

Dashboard phải có ít nhất các panel:

1. Vibration RMS theo từng máy.
2. Temperature theo từng máy.
3. Current ampere theo từng máy.
4. Health score theo thời gian.
5. Trạng thái máy: `running`, `stopped`, `alarm`.
6. Số event theo loại event.
7. Bảng event gần nhất.

10. Tiêu chí chấm điểm riêng

Nội dung	Điểm
Kiến trúc machine health gateway hợp lý	1.0
Sensor mô phỏng được dữ liệu máy và fault scenario	1.2
Actuator nhận lệnh và publish trạng thái đúng	1.0
Gateway validate, normalize và lưu state đúng	1.0
Thiết kế health score có giải thích hợp lý	1.2
Rule engine phát hiện vibration, overheat, over-current và dừng máy	1.4
InfluxDB và Grafana thể hiện health monitoring rõ ràng	1.1
REST API hoạt động đúng	0.8
Docker Compose, README, báo cáo, demo tốt	1.3
Tổng	10.0

Đề tài 5. Virtual Smart Energy Management Gateway

1. Tên đề tài

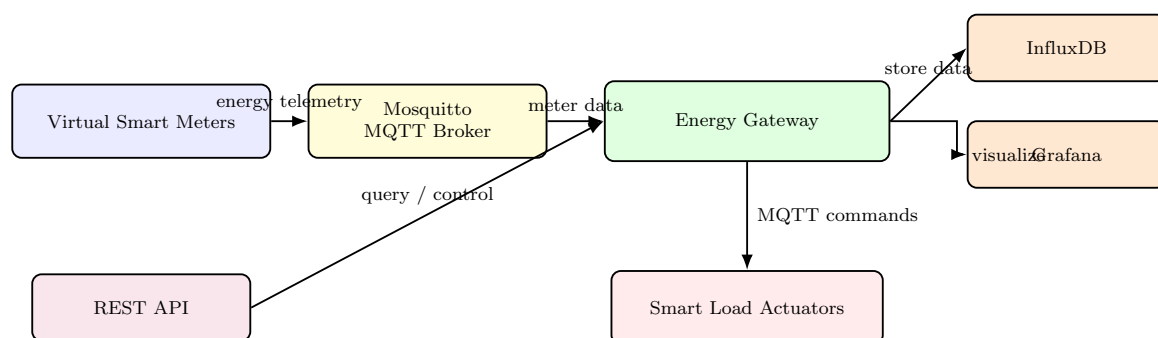
Xây dựng hệ thống Virtual Smart Energy Management Gateway cho giám sát điện năng và điều khiển tải thông minh qua MQTT

2. Bối cảnh và mục tiêu

Trong nhà thông minh hoặc tòa nhà nhỏ, hệ thống quản lý năng lượng cần giám sát tiêu thụ điện của nhiều tải như điều hòa, đèn, máy bơm, ổ cắm thông minh và nguồn năng lượng mặt trời giả lập. Gateway có thể phát hiện tổng công suất vượt ngưỡng, lập lịch tắt bớt tải không ưu tiên, cảnh báo bất thường và hiển thị dữ liệu tiêu thụ điện theo thời gian.

Đề tài này yêu cầu sinh viên xây dựng một smart energy gateway ảo. Các smart meter và load actuator được giả lập bằng Python. Gateway phải thu thập dữ liệu công suất, tính tổng điện năng tiêu thụ, phát hiện overload, ra quyết định điều khiển tải và cung cấp API để giám sát hoặc điều khiển thủ công.

3. Kiến trúc hệ thống



Hình 5: Kiến trúc hệ thống của Đề tài 5

4. Các service bắt buộc

- mosquitto: MQTT broker.
- meter-hvac, meter-lighting, meter-plug: smart meter giả lập cho các nhóm tải.
- load-hvac, load-lighting, load-plug: actuator giả lập.
- solar-simulator: nguồn năng lượng mặt trời giả lập, tùy chọn nhưng khuyến khích.
- energy-gateway: gateway quản lý năng lượng.
- energy-api: REST API.
- influxdb: lưu dữ liệu time-series.

- grafana: dashboard giám sát.

5. MQTT topic gợi ý

```
energy/hvac/meter/telemetry
energy/lighting/meter/telemetry
energy/plug/meter/telemetry
energy/solar/telemetry

energy/hvac/load/command
energy/lighting/load/command
energy/plug/load/command

energy/hvac/load/status
energy/lighting/load/status
energy/plug/load/status

energy/gateway/event
energy/gateway/summary
```

6. Message format yêu cầu

Meter telemetry:

```
{
  "device_id": "meter-hvac",
  "load_id": "hvac",
  "voltage": 220.0,
  "current_ampere": 5.6,
  "power_watt": 1232.0,
  "energy_wh": 340.5,
  "priority": "low",
  "timestamp": "2026-06-12T10:00:00Z"
}
```

Gateway summary:

```
{
  "total_power_watt": 3650.0,
  "solar_power_watt": 850.0,
  "grid_power_watt": 2800.0,
  "overload": true,
  "timestamp": "2026-06-12T10:00:05Z"
}
```

Command message:

```
{
  "load_id": "plug",
  "target": "load_switch",
  "action": "off",
  "reason": "overload_shedding",
  "timestamp": "2026-06-12T10:00:06Z"
}
```

7. Logic gateway và rule engine

Gateway phải có ít nhất 5 luật xử lý:

1. Tính `total_power_watt` bằng tổng công suất của các tải đang bật.
2. Nếu `total_power_watt` lớn hơn 3500, sinh event `overload_detected`.
3. Khi overload, tắt tải có priority thấp trước, ví dụ plug trước lighting, và không tắt tải priority cao nếu chưa cần thiết.
4. Nếu `solar_power_watt` cao, ưu tiên bật lại một số tải đã bị tắt trước đó.
5. Nếu một smart meter không gửi dữ liệu trong một khoảng thời gian cấu hình, sinh event `meter_offline`.

Gateway phải publish `energy/gateway/summary` định kỳ để lưu và hiển thị tổng quan năng lượng.

8. REST API tối thiểu

```
GET /health
GET /loads
GET /loads/hvac/state
GET /energy/summary
GET /events
POST /loads/plug/command
```

API phải cho phép bật hoặc tắt thủ công một tải, đồng thời có endpoint trả về tổng công suất hiện tại.

9. Dashboard Grafana

Dashboard phải có ít nhất các panel:

1. Công suất từng tải theo thời gian.
2. Tổng công suất tiêu thụ.
3. Công suất solar giả lập.
4. Grid power theo thời gian.
5. Trạng thái bật tắt của từng tải.
6. Số event overload theo thời gian.
7. Bảng event gần nhất.

10. Tiêu chí chấm điểm riêng

Nội dung	Điểm
Kiến trúc smart energy gateway hợp lý	1.0
Smart meter simulator sinh dữ liệu hợp lý	1.0
Load actuator nhận command và publish status đúng	1.0
Gateway tính tổng công suất, grid power và lưu state đúng	1.3
Rule engine overload shedding và khôi phục tải hợp lý	1.5
InfluxDB và Grafana thể hiện power, event, load status rõ ràng	1.2
REST API hoạt động đúng	0.8
Docker Compose, Dockerfile, volume, environment variables đúng	1.0
README, báo cáo, demo và giải thích tốt	1.2
Tổng	10.0